Politecnico di Milano

Ingegneria del Software – a.a. 2006/07

Appello del 14 settembre 2007

Cognome

Nome

Matricola

Sezione (segnarne una) Baresi ☐, Ghezzi ☐, Morzenti ☐, SanPietro ☐

Istruzioni

1. La mancata indicazione dei dati anagrafici e della sezione comporta l'annullamento del compito.
2. Al termine, consegnare **solo i fogli distribuiti** utilizzando il **retro** delle pagine in caso di necessità. Non separare questi fogli. Eventuali fogli di brutta, ecc. **non** verranno **in nessun caso** presi in considerazione. È possibile scrivere **in matita**.
3. È possibile consultare liberamente libri, manuali o appunti. È **proibito** l'uso di ogni dispositivo elettronico (quali calcolatrici tascabili, telefoni cellulari, ecc.).
4. Non è possibile lasciare l'aula conservando il tema della prova in corso.
5. Tempo a disposizione: 2h.
6. Punteggio totale a disposizione: 25/30. La sufficienza si raggiunge con 16 punti, a cui sommare il risultato delle prove di laboratorio.

Esercizio 1: .....

Esercizio 2: .....

Esercizio 3: .....

Esercizio 4: .....

Totale .....

Esercizio 1 (punti 6)

In presenza della seguente dichiarazione

```
interface Functions { //funzioni a valore float di una variabile float
   float  value (float x); // calcola il valore di this in corrispondenza al parametro x
}
```

il metodo

public static float [] findFunctionZeros(functions f, float a, float b, char p) {... }

accetta come ingresso un oggetto *Functions* (assunto non nullo), due parametri *a* e *b* che individuano un intervallo non vuoto e un carattere *p* che può valere 'l' (per indicare "bassa precisione") oppure 'h' ("alta precisione") e restituisce un array di float che contiene valori che rappresentano zeri della funzione *f* compresi nell'intervallo tra *a* e *b*: se p=='l' allora il valore della funzione, per ognuno dei valori memorizzati nell'array restituito, è in valore assoluto <0.001; se p=='h' è <0.000001.

Si scriva una specifica, in JML del metodo, indicando la pre- e la post-condizione.

Soluzione

```
//@requires f!= null && a<=b && (p=='l' || p=='h')

//@ensures
//@ \result !=null &&
//@ (\forall int i; 0<=i && i<\result.length;
//@  \result[i]>=a && \result[i]<=b &&
//@  f.value(\result[i])> (p=='l'?-0.001:-0.000001) && f.value(\result[i])< (p=='l'?0.001:0.000001) )
```

Esercizio 2 (punti 12)

La classe **Grafo** rappresenta grafi i cui archi sono orientati e pesati e la sua interfaccia comprende il metodo *observer* **ArrayList<Nodo> nodi()**, che restituisce tutti i nodi del grafo. L'interfaccia della classe **Nodo** comprende i seguenti metodi *observer*:

- **Arco arco(Nodo dest)** restituisce l'arco che collega this a dest, se esiste ; null altrimenti.
- **ArrayList<Arco> archi(int dir)** restituisce gli archi che entrano (dir = 0) o escono (dir =1) dal nodo.
- **boolean raggiungibile(Nodo dest)** restituisce true se dest è raggiungibile da this e false altrimenti.

L'interfaccia della classe **Arco** comprende i seguenti metodi *observer*:

- **Nodo sorgente()** restituisce il nodo sorgente dell'arco considerato
- **Nodo destinatario()** restituisce il nodo destinatario dell'arco considerato
- **int peso()** restituisce il peso dell'arco.

a. Si scrivano la pre- e post-condizione del metodo **void aggiungiArco(Nodo dest, int peso) della** classe **Nodo**, sapendo che_ l'oggetto this è il nodo sorgente dell'arco, una coppia di nodi può essere connessa da un solo arco e il peso del nuovo arco deve essere positivo. Si ricorda anche che dopo l'aggiunta, il nuovo arco deve comparire nell'insieme di archi che entrano/escono dai nodi passati come parametri.

```
// dest diverso da null
//@ requires dest != null;
// peso nuovo arco positivo
//@ requires peso > 0;
// non esiste un arco da this a dest
//@ requires !(\exists int i; 0 <= i <= archi(1).size(); archi(1).get(i)==dest)

// esiste un arco che esce da this e entra in dest
//@ ensures (\exists int i; 0 <= i <= archi(1).size(); archi(1).get(i).destinatario()==dest)
// esiste un arco che entra in dest da this
//@ ensures !(\exists int i; 0 <= i <= dest.archi(0).size(); dest.archi(0).get(i).sorgente()==this)

void aggiungiArco(Nodo dest, int peso)
```

b. Si scriva la post-condizione del metodo **void rimuoviArco(Arco a),** supponendo che appartenga alla classe **Grafo.** Si noti che un nodo che rimane isolato dopo la cancellazione dell'arco, deve pure essere cancellato.

```
/l'arco non deve piu' esistere
//@ ensures (\forall int i; 0 <= i <= nodi().size();
    ((\forall int j; 0 <= j < nodi().get(i).archi(1).size();
       nodi().get(i).archi(1).get(j) != a) &&
     (\forall int j; 0 <= j < nodi().get(i).archi(0).size();
       nodi().get(i).archi(0).get(j) != a));
// non ci devono essere nodi isolati
//@ ensures (\forall int i; 0 <= i <= nodi().size();
    nodi().get(i).archi(1).size() >0 || nodi().get(i).archi(0).size() > 0);
```

c. Si scriva in JML l'invariante pubblico della classe **Grafo** sapendo che un grafo non può mai avere nodi isolati e archi senza sorgente o destinatario.

```
//@ public invariant
// non ci devono essere nodi isolati
(\forall int i; 0 <= i < nodi().size();
    nodi().get(i).archi(1).size() >0 || nodi().get(i).archi(0).size() > 0) &&
// non ci devono essere archi senza sorgente o destinatario
(\forall int i; 0 <= i < nodi().size();
    ((\forall int j; 0 <= j < nodi().get(i).archi(1).size();
       nodi().get(i).archi(1).get(j).destinatario() != null) &&
     (\forall int j; 0 <= j < nodi().get(i).archi(0).size();
       nodi().get(i).archi(0).get(j) != null));
```

d. Si scrivano la pre- e post-condizione del metodo int distanzaMinima(Nodo dest) della classe **Nodo** che calcola la distanza minima tra l'oggetto this e il nodo dest, se dest è raggiungibile da this. Si consiglia di adottare una definizione ricorsiva del problema per semplificare la scrittura della post-condizione.

```
// il nodo deve essere raggiungibile
//@ requires raggiungibile(dest);

// se i due nodi sono connessi direttamente, la distanza minima e' il
// peso dell'arco che connette i due nodi, altrimenti dobbiamo
// identificare ogni nodo p collegato a this da un arco e la distanza
// minima e' il minimo tra le somme della distanza tra this e p e
// p.distanzaMinima(dest).
// Per semplicità ipotizzo che 1000 sia un valore fuori scala.

//@ ensures (\exists int i; 0 <= i < archi(1).size(); archi(1).get(i).destinatario() == dest)?
\result == arco(dest).peso():
\result == (\min int i; 0<= i < archi(1).size(); archi(1).get(i).destinatario().raggiungibile()?
                arco(archi(1).get(i).destinatario()).peso() +
               archi(1).get(i).destinatario().distanzaMinima(dest):1000);

int distanzaMinima(Nodo dest)
```

Esercizio 3 (punti 3)

Si implementi un iteratore per la classe **Grafo,** dell'esercizio precedente, in modo che dato un nodo n, l'iteratore restituisca tutti i nodi raggiungibili da n dal più vicino al più lontano. Si ipotizzi che esista un metodo **int distanza(Nodo dest)** della classe Nodo che restituisce la distanza tra this e un nodo dato come parametro.

```
public class Grafo {
  ...
  public Iterator<Nodo> nodiRaggiungibili(Nodo n) {
    return new GenNodi(n);
  }

  //classe interna
  private static class GenNodi implements Iterator<Nodo> {
    private int pos; // posizione corrente
    private ArrayList<Nodo> nodiOrdinati; // struttura per I nodi ordinati

    GenNodi(Nodo n) {
      pos=0;
    // nodiOrdinati riempita in funzione di n.archi() e dei loro destinatari
    }

    public boolean hasNext() {return pos < nodiOrdinati().size()}

    public int next() throws NoSuchElementException {
      if (nodiOrdinati.size() > 0 && pos < nodiOrdinati.size())
        return nodiOrdinati.get(pos++);
      throw new NoSuchElementException();
    }
}
```

Esercizio 4 (punti 4)

Si definisca un diagramma delle classi UML per rappresentare il seguente problema descrivendo con accuratezza le relazioni fra le classi e le loro cardinalità, ove necessario.

Una banca si vuole dotare di un nuovo sistema informativo per unificare la gestione dei prodotti bancari. Nello specifico, la banca tratta conti correnti tradizionali, conti correnti online, mutui, e portafogli titoli. Mentre i conti correnti sono sottoscrivibili da tutti, i mutui e i portafogli titoli sono riservati ai clienti intestatari di un conto corrente con una giacenza media di 2000 euro negli ultimi tre mesi.
Ciascun cliente della banca può scegliere di aprire alternativamente un conto corrente online, uno tradizionale, ma non entrambi. Il conto corrente online prevede esclusivamente l'accesso al conto tramite il sito Internet ed è necessariamente associato ad una carta Bancomat che è invece opzionale nel caso del conto tradizionale. Entrambi i conti possono poi essere associati a carta di credito e libretto assegni. Anche Carta di credito e libretto assegni sono riservati ai clienti titolari di un conto corrente con una giacenza media di 2000 euro negli ultimi tre mesi.
Nel caso del conto online, il versamento di denaro può avvenire solo tramite assegno o bonifico, mentre nel caso del conto tradizionale è anche possibile versare contanti. I conti correnti sono associati a delle spese fisse, e a delle spese per ciascuna operazione sul conto (prelievo, versamento, ecc.). Tuttavia sono previsti dei bonus per i clienti che effettuano un numero elevato di operazioni mensili.